

MILESTONE 4 – INTEGRATION, AUTHENTICATION, DASHBOARD & ALERTING

Sentinel Core Enterprise Operation Platform

Milestone Completion Report

1. Milestone Objective

The objective of Milestone 4 was to integrate the Sentinel Core application into a cohesive working system. The expected output was an application in which a user can log in, access a live dashboard according to their role, remain authenticated seamlessly through token refresh, and use the complete alerting pipeline from health check to alert generation and email notification.

2. Expected Final Output

A fully integrated app where a user logs in, sees a live dashboard scoped to their role, stays logged in seamlessly across token refreshes, and the whole alerting pipeline (health check → alert → email) fires correctly end to end — completing Milestone 4 as a cohesive, working system.

3. Scope of Work Completed

Area	Work Completed	Outcome
User Authentication	JWT-based login and protected application access were integrated.	Users can securely enter the application.
Role-Based Access	Dashboard and administrative actions are controlled according to the user's role.	Users receive access appropriate to their role.
Token Refresh & Session Continuity	Authentication handling was improved so users can remain logged in when access tokens expire and refresh succeeds.	Session continuity is improved during token expiry.
Live Dashboard	React dashboard was connected to backend APIs and database-backed asset information.	Operational information is retrieved through APIs.
System Health Metrics	CPU, memory, disk usage and uptime were integrated into dashboard health cards.	Database-backed resource metrics are displayed.
Dashboard Refresh	Periodic API refresh was introduced so operational values can update without a full page reload.	Dashboard values can update periodically.
Alert Monitoring	Backend alert information is surfaced through the dashboard.	Operational alerts are visible.
Health Check → Alert	Health/status conditions feed the alert-generation workflow.	Health conditions can trigger alerts.
Email Notification	The alert workflow is connected to email notification.	Configured alerts can notify users by email.

UI/UX Improvements	Metric cards, status indicators, search/filtering, alerts and role-aware controls were improved.	The monitoring interface is clearer and more usable.
--------------------	--	--

4. System Architecture

The milestone integrates the following layers:

- React Frontend – login, dashboard, asset monitoring, alerts and role-aware UI.
- Spring Boot Backend – REST APIs, authentication/authorization, asset and alert processing.
- PostgreSQL Database – users, assets, health metrics and operational information.
- JWT Authentication – secure access to protected APIs and authenticated sessions.
- Alerting Layer – detects abnormal or critical health/status conditions.
- Email Notification Layer – delivers notifications for configured alert conditions.

Overall flow: User Login → JWT Authentication → Role Validation → Dashboard → Backend API → PostgreSQL Data → Health Evaluation → Alert Generation → Email Notification.

5. Authentication and Authorization

Milestone 4 strengthened the JWT-based security flow between the React frontend and Spring Boot backend. Protected APIs require valid authentication, while the frontend maintains authenticated state and handles token lifecycle events.

- User submits login credentials to the backend.
- Backend validates the user and issues JWT authentication information.
- Frontend attaches authentication information to protected API requests.
- Role information controls dashboard functions and administrative actions.
- Token refresh handling reduces unnecessary logout and login redirects.

6. Role-Scoped Dashboard

The dashboard was integrated with role-aware behavior. Administrative controls can be restricted to users with the required role, while viewer users can access monitoring information without receiving unauthorized administrative controls. Backend authorization remains the security boundary, while the frontend provides the corresponding user experience.

7. Live Dashboard Integration

The dashboard retrieves operational information from the backend rather than depending only on hard-coded values. The dashboard summary API provides total assets, uptime, online/offline status, critical information and average resource utilization.

- Total Assets
- Online and Offline Assets
- Uptime Percentage
- CPU Usage
- Memory Usage

- Disk Usage
- Warning/Critical Status
- Alerts

A frontend/backend field-mapping issue was identified for resource metrics. The backend exposes avgCpuUsage, avgMemoryUsage and avgDiskUsage; the frontend was aligned to consume these exact API fields so database-backed values are displayed instead of incorrectly falling back to zero.

8. Dashboard Refresh and UI/UX

- CPU, memory and disk usage are presented through dedicated health/metric cards.
- System status messaging is dynamically based on operational conditions.
- Periodic refresh retrieves updated API values without a complete browser reload.
- Asset search/filtering improves navigation.
- Alerts are surfaced directly within the dashboard.
- Role-specific controls are hidden or restricted where appropriate.
- The layout is organized for quick operational monitoring.

9. End-to-End Alerting Pipeline

The intended alerting pipeline is integrated as an operational flow:

1. Health Check – system or asset health information is evaluated.
2. Condition Detection – abnormal or critical conditions are identified.
3. Alert Creation – the backend creates an alert with asset, severity and message information.
4. Alert Exposure – alert data is available through the API and dashboard.
5. Email Notification – configured alert conditions can trigger email delivery.
6. User Visibility – the dashboard makes the operational condition visible.

10. API and Data Integration

The milestone required alignment between frontend API consumption, backend service logic and PostgreSQL persistence. The dashboard summary endpoint aggregates asset information before returning it to React.

- totalAssets
- uptimePercentage
- onlineAssets
- offlineAssets
- criticalAlerts
- avgCpuUsage
- avgMemoryUsage
- avgDiskUsage

11. Testing and Validation

Test Area	Validation	Expected Result
Login	Valid credentials tested	User enters application
Protected Dashboard	Authenticated dashboard access reviewed	Protected functionality requires authentication
Role-Based UI	Admin/viewer behavior reviewed	Administrative controls appear only where permitted
Token Lifecycle	Expiry/refresh behavior reviewed	User remains authenticated when refresh succeeds
Dashboard API	API response and frontend mapping checked	Backend values appear in UI
Resource Metrics	CPU, memory and disk fields checked	Non-zero database values display correctly
Periodic Refresh	Polling behavior reviewed	Updated values appear without full reload
Alerts	Alert data displayed	Operational issues are visible
Email Flow	Alert-to-notification integration reviewed	Configured alert conditions can trigger email

12. Issues Identified and Resolved

Issue	Resolution
CPU/Memory/Disk displayed as 0%	Frontend property names did not match backend response fields. Mapping was aligned to avgCpuUsage, avgMemoryUsage and avgDiskUsage.
Session redirected to login after token expiry	Token refresh/session handling was integrated to maintain authentication when refresh succeeds.
Static system-health message	Status messaging was made dynamic based on warning/critical conditions.
Need for continuously updated monitoring	Periodic dashboard refresh was introduced.

13. Technologies Used

- Frontend: React, JavaScript, Material UI and Axios
- Backend: Java, Spring Boot and REST APIs
- Database: PostgreSQL
- Security: JWT authentication and role-based authorization
- Version Control: Git and GitHub
- Notification: Email-based alert notification

14. Milestone 4 Outcome

Milestone 4 brings the major Sentinel Core components together into a unified monitoring workflow. The application supports authenticated access, role-aware dashboard behavior, database-backed operational metrics, periodic dashboard updates, alert visibility and the health-check-to-email notification architecture.

Integration defects discovered during validation were addressed, particularly the mismatch between backend dashboard metric names and frontend consumption. As a result, CPU, memory and disk utilization can be displayed from the backend/database data rather than incorrectly defaulting to zero.

15. Conclusion

Milestone 4 was completed with the objective of delivering a cohesive Sentinel Core application. The work connects authentication, authorization, dashboard monitoring, database-backed metrics and alerting into a single user workflow. The resulting system provides a foundation for enterprise asset monitoring and can be extended with real-time telemetry, advanced alert rules, notification preferences and production-grade observability.